TRABAJO PRÁCTICO N.º 8 (UNIDAD 8) ? FOOD STORE EN LA FERIA

Asignatura: Base de Datos II · UTN - Tecnicatura Universitaria en Programación · Docente: Carlos Yácomo
Grupo H (Saferazi): Lautaro Fernandez, Daniela Díaz, Jesús Ramírez, Danilo Serrano, Elio Marí
Entorno: PostgreSQL 17 (SSOT), SQLite 3.50+ (STRICT), MongoDB 8.0 · Etiqueta de entrega: tp8

REGLA DE ORO DE LA MATERIA:
El móvil propone, el servidor dispone. La tablet solo consulta y arma borradores. Precio, stock y ventas los decide PostgreSQL.

1. CONSIGNA 1.a · QUÉ DATOS VIAJAN A LA TABLET Y FUENTE DE VERDAD

Tabla Original	Columna / Atributo	¿Viaja?	Justificación Técnica, Operativa y de Seguridad
`producto`	id, nombre	**SÍ**	Identificación unívoca y visualización en el menú de la tablet.
`producto`	precio -> precio_centavos	**SÍ**	Centavos enteros (ROUND(precio*100)) evitando errores de float.
`producto`	stock -> stock_visto	**SÍ**	Referencia visual para que el cajero conozca el stock al inicio del turno.
`producto`	disponible, eliminado	**SÍ**	Patrón de lápidas (tombstones): filas eliminadas quedan ocultas.
`producto`	categoria_id, imagen, meta	**NO**	Prescindibles para facturar; ahorran ancho de banda y almacenamiento.
`usuario`	contrasena, mail, rol, tel	**NO**	Zero Trust: Tablet pública expuesta a hurto; credenciales nunca viajan.
`pedido / detalle`	Histórico de ventas previo	**NO**	La tablet solo gestiona borradores de la feria actual, no transacciones ajenas.

Por qué y Fuente de Verdad:
A la tablet solo viaja una copia de lectura podada del catálogo (precios y stock referencial), descartando claves de usuarios, datos personales y metadatos internos. Esto reduce el consumo de datos móviles en ferias y previene filtraciones ante robo físico. La única fuente de verdad (Single Source of Truth - SSOT) es el servidor PostgreSQL. La tablet opera con un caché local y solo **propone borradores; precio vigente, control de inventario y confirmación contable son potestad exclusiva de PostgreSQL.**

2. CONSIGNA 2.c · DEMOSTRACIÓN DE CASOS DE SINCRONIZACIÓN Y SEMÁFORO

El algoritmo del semáforo evalúa solicitudes en 'PENDIENTE' cruzando con snapshot_servidor con orden de prioridad:
1. ROJO (RECHAZADO): Quiebre de stock, producto dado de baja o inexistente.
2. AMARILLO (CONFLICTO): Stock suficiente, pero el precio en el servidor difiere del visto en el borrador.
3. VERDE (SIMULADO_APROBADO): Coincidencia plena. Aprobación simulada (no modifica PostgreSQL).

Caso C2 · Suba de precio en servidor -> CONFLICTO -> Aceptación del cliente -> SIMULADO_APROBADO
Preparación: Borrador con 2 u. de Fugazzeta a 180.000 centavos ($1.800). En servidor sube el precio a 210.000 centavos ($2.100).
`Comando ejecutado: UPDATE snapshot_servidor SET precio_centavos = 210000 WHERE id = 1; (y semaforo.sql)`
Resultado 1 (req-c2-v1): Estado: CONFLICTO | Motivo: 'CONFLICTO: Precio modificado en el servidor respecto al borrador'
Resolución: Cliente acepta suba. Se actualiza precio del borrador a 210.000 centavos y se emite la solicitud 'req-c2-v2'.
Resultado 2 (req-c2-v2): Estado: SIMULADO_APROBADO | Motivo: 'SIMULADO_APROBADO: Stock y precios validados contra la instantanea'

2. CASOS DE SINCRONIZACIÓN (CONTINUACIÓN: C3 A C6)

Caso C3 · Borrador por más unidades de las que quedan -> RECHAZADO

Preparación: Borrador 'b-c3-001' solicita 30 unidades de Fugazzeta. En el servidor solo quedan 15 unidades en inventario.

`Comando ejecutado: INSERT INTO item_borrador VALUES ('b-c3-001', 1, 30, 180000); (y semaforo.sql)`

Resultado (req-c3-001): Estado: RECHAZADO | Motivo: 'RECHAZADO: Stock insuficiente en el servidor'

Explicación: La condición (s.stock < i.cantidad) gatilla prioridad roja, frenando la solicitud e impidiendo comprometer stock faltante.

Caso C4 · Producto dado de baja lógica en servidor -> RECHAZADO y Patrón de Lápidas

Preparación: Borrador con Jamón y Queso (id 3). Servidor aplica baja lógica marcando eliminado = 't' y disponible = 'f'.

`Comando: UPDATE snapshot_servidor SET eliminado = 't', disponible = 'f' WHERE id = 3; (y semaforo.sql)`

Resultado (req-c4-001): Estado: RECHAZADO | Motivo: 'RECHAZADO: Producto no disponible o dado de baja'

Patrón de Lápidas: Al recargar catálogo local, la fila persiste en producto_cache con eliminado = 1 (tombstone) evitando roturas de claves foráneas históricas, pero deja de figurar automáticamente en las consultas del catálogo activo comercial.

Caso C5 · Borrador coincidente sin cambios -> SIMULADO_APROBADO

Preparación: Borrador con 2 unidades de Mozzarella a 160.000 centavos ($1.600). En servidor hay 20 u. al mismo precio.

`Comando: INSERT INTO item_borrador VALUES ('b-c5-001', 2, 2, 160000); (y semaforo.sql)`

Resultado (req-c5-001): Estado: SIMULADO_APROBADO | Motivo: 'SIMULADO_APROBADO: Stock y precios validados contra la instantanea'

Explicación: Coincidencia plena. Factible pero no constituye una venta real en PostgreSQL.

Caso C6 · Reenvío de la misma solicitud e Idempotencia (Mismo UUID)

Preparación: Reinsertar solicitud_uuid duplicado ('req-c5-001') simulando reintento descontrolado por micro-corte de señal.

`Comando: INSERT INTO cola_sync VALUES ('req-c5-001', 'b-c5-001', 'PENDIENTE', ...)`

Resultado SQLite: UNIQUE constraint failed: cola_sync.solicitud_uuid (Falla por Clave Primaria).

Comportamiento del Semáforo: Re-ejecutar el semáforo no reevalúa solicitudes ya resueltas (filtra WHERE estado = 'PENDIENTE').

Dónde iría la protección real en un sistema de verdad (Parada 8):

En producción la protección se ubica obligatoriamente en PostgreSQL con una restricción UNIQUE sobre 'idempotency_key' en la tabla de órdenes; ante reenvíos de red, el backend detecta el UUID y responde con HTTP 200 sin duplicar cobros ni stock.

Verificación de Inmutabilidad en PostgreSQL (Servidor Intacto)

Tras correr todos los casos móviles en SQLite, se ejecutó en la base central PostgreSQL:

`SELECT (SELECT count(*) FROM foodstore.pedido) AS total_pedidos, (SELECT sum(stock) FROM foodstore.producto) AS stock_total`

Resultado: Pedidos = 3 (sin cambios) | Stock Total = 261 unidades (exactamente idéntico al valor previo).

Conclusión: Se demuestra de forma fehaciente que SIMULADO_APROBADO no modificó el stock ni creó ventas en el servidor.

3. CONSIGNA 3.a · MAPA DE LA CAPA DE DATOS (LOS TRES MOTORES)

Motor y Versión	Rol Arquitectónico	¿Qué guarda cada uno?	¿Quién manda sobre cada dato?
PostgreSQL 17	Núcleo Transaccional (OLTP) Relacional ACID	Catálogo maestro (producto, categoria), usuarios (hashes contraseñas), pedidos oficiales confirmados y stock real central.	**FUENTE DE VERDAD (SSOT):** Autoridad suprema e indelegable sobre precios maestros, stock y ventas válidas.
MongoDB 8.0	Lectura y Analítica (U7) Documental NoSQL	Colección 'pedidos': documentos JSON desnormalizados con cliente y arreglo de líneas de productos embebidas.	**SUBORDINADO A POSTGRESQL:** Proyección de lectura aguas abajo. No tiene autoridad sobre stock ni precios.
SQLite 3.50+	Capa Móvil Offline (U8) Relacional Embebido	producto_cache (catálogo local con lápidas), borradores locales (pedido_borrador, ítems) y cola de sincronización (cola_sync).	**MANDA SÓLO EN BORRADORES:** Autoridad local sobre sus borradores. CERO potestad sobre servidor.

4. CONSIGNA 3.b · PLAN DE MANTENIMIENTO (ADENDA A LA UNIDAD 6)

1. ¿Qué se respalda y cómo se prueba la restauración?

- Servidor PostgreSQL: Respaldo lógico diario con pg_dump -Fc (formato custom con metadatos y checksums). La restauración se prueba semanalmente de forma automatizada sobre base descartable (foodstore_restore_test), midiendo el RTO y validando conteos (U6).
- Tablets Móviles SQLite: La base es efímera. producto_cache se descarga bajo demanda desde PostgreSQL. Los borradores locales no sincronizados se respaldan localmente en copia física de foodstore_movil.db y log JSON en tarjeta flash antes de sincronizar.

2. Si se restaura un respaldo viejo de PostgreSQL, ¿qué hacen las tablets?

- Detección de Desfasaje: Las tablets detectan un epoch/timestamp del servidor anterior a su copia local al contactar la API.
- Protocolo de Recuperación: 1) Invalidan de inmediato su producto_cache; 2) Descargan la instantánea limpia del servidor restaurado; 3) Re-ejecutan forzadamente el semáforo sobre borradores no consolidados. Pedidos previamente en SIMULADO_APROBADO con precios o stock desfasados pasan a CONFLICTO o RECHAZADO, alertando al cajero para revalidar con el cliente.

3. ¿Cada cuánto y quién revisa las solicitudes en CONFLICTO o RECHAZADO?

- En Tiempo Real (Punto de Venta): El cajero u operador del food truck resuelve la alerta al instante en el mostrador (consulta al cliente si acepta el aumento ante CONFLICTO de precio, o sugiere reemplazos si hay quiebre de stock).
- Al Cierre de Turno (Diario): El Supervisor de Operaciones audita el reporte consolidado de incidencias en la cola central para identificar desajustes crónicos entre stock físico y teórico, o caídas repetitivas de red móvil.

5. CONSIGNA 3.c · CODE REVIEW CON IA SOBRE SCRIPTS DE LA CURSADA

Revisión sistemática de scripts con OpenCode / IA en agente Plan. Se presenta la matriz de decisiones y comprobación:

Unidad / Script	Hallazgo Detectado	Decisión	Comprobación en Base y Justificación

U1 (Relacional)
`schema.sql`
`objects.sql`
	En detalle_pedido, subtotal se calcula por trigger pero la colum ero carece de CHECK de consistencia ante updates directos.	**Aceptado y Corregido**	Comprobado en PostgreSQL: Se incorporó CHECK (subtotal = * precio_unitario). Se verificó que el motor aborta intentos de in tar subtotales artificiales.

U6 (Backups)
`restore_test.ps1`
`restore_test.sh`
	La IA propuso forzar pg_restore --clean --if-exists contra la a tendida en producción para 'recuperar automáticamente' ante fallos'.	**Descartado con Fundamento**	DESCARTE CRÍTICO: Ejecutar DROP DATABASE forzado en va ante alertas espurias destruye sesiones activas. Solo se aut en staging (foodstore_restore_test); en prod requiere corte forn autorización humana.

U7 (NoSQL)
`consultas_mongo.js`
`laboratorio_ia.js`
	Los importes en MongoDB se insertaban como Double, provocan do pérdida de centavos por redondeo IEEE 754.	**Aceptado y Corregido**	Comprobado en MongoDB: Se migró a Decimal128 (NumberDe validador BSON, verificando acumulados contables exactos sin

U8 (SQLite) `schema_sqlite.sql`	SQLite por defecto permite tipado dinámico debilitando la pre itiendo almacenar texto en columnas monetarias.	**Aceptado y Corregido**	Comprobado en SQLite: Se forzó cláusula STRICT en las 4 tab erificando que rechaza strings en columnas INTEGER inmediat

6. SÍNTESIS Y CONCLUSIONES ARQUITECTÓNICAS

Cierre de la Capa de Datos del Proyecto Food Store:

1. Desacoplamiento Efectivo: La aplicación móvil opera de forma totalmente autónoma frente a cortes de conectividad gracias al catálogo local en SQLite STRICT, utilizando enteros en centavos para eliminar cualquier imprecisión de punto flotante en el borde.
2. Semáforo Idempotente: La evaluación con prioridades (Rojo > Amarillo > Verde) previene que pedidos inviables saturen la red central, mientras que el filtrado por solicitudes PENDIENTE garantiza que la repetición de la simulación no altere registros históricos.
3. **Respeto Irrestricto a la Regla de Oro: Se demostró empíricamente que SIMULADO_APROBADO no es una venta en PostgreSQL. La integridad, el stock real y el balance contable permanecen bajo la custodia absoluta de PostgreSQL como Single Source of Truth.**